

P1207R1

Movability of Single-pass Iterators

Published Proposal, 2019-01-21

Author:

Corentin Jabot

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose move-only non-forward iterators in the ranges namespace along with a refined, tag-less iterator taxonomy with the express intent to increase safety and expressiveness of programs handling input iterators.

Table of Contents

1	Changes
1.1	Revision 1
2	Introduction
3	Terminology
4	Scope
4.1	Non-Goal
5	Motivation
5.1	Move-only state
5.2	Implicitly destructive operations
6	What is a move-only iterator?
7	A Holistic Approach to Iterators
7.1	An Holistic Approach to Iterator Tags and Iterator Concepts
7.1.1	Tags as an opt-in opt-out mechanism
8	Q/A
8.1	Non-regular iterators, really?
8.2	What about Equality of Input Iterators?
8.3	But... Moved-from objects are still objects!
8.4	Does iterators default-constructability needs revisiting?
8.5	What about [P0902R0]?
8.6	Why do you want to take my Copyable InputIterators away from me, I like them?!
8.7	Will this break existing code ?!
8.8	Won't that implicit categorization lead to miss-categorization?
8.9	Post Increment on non-copyable iterators
9	Questions for LEWG
10	List of proposed changes
10.1	Changes to <iterator>
10.1.1	WeaklyIncrementable
10.1.2	Iterator
10.1.3	InputIterator
10.1.4	OutputIterator
10.1.5	ContiguousIterator
10.2	Changes to <ranges>
10.2.1	Views
10.2.2	Inserters
10.2.2.1	back_insert_iterator
10.2.2.2	front_insert_iterator
10.2.2.3	insert_iterator
10.2.3	counted_iterator
10.2.4	move_iterator
10.3	Changes to <algorithm>
11	Impact on other proposals
11.1	View proposals

11.2 Iterator facade

12 Implementation experience

12.1 Acknowledgments

References

Informative References

I want to move(it), move(it), y'all want to move(it);

§ 1. Changes

§ 1.1. Revision 1

- Refine the impact on the standard
- Mention `ITER_CONCEPT`
- Remove the idea the `ranges::copy` could move from non-copyable iterators
- Replace Cpp17Iterator by LegacyIterator to reflect the standard
- Remove the very wrong idea of having view return `begin()` by reference.
- Fix some confusing phrasing

§ 2. Introduction

Non-forward Input iterators and output iterators, also known as "Single-pass iterators" are semantically move-only. The standard states:

Note: For input iterators, $a == b$ does not imply $++a == ++b$ (Equality does not guarantee the substitution property or referential transparency.) Algorithms on input iterators should never attempt to pass through the same iterator twice. They should be single pass algorithms.

This means that once an iterator is copied, only one of the copies can be read-from if either one is incremented, which make the usefulness of such object questionable. Deferencing multiple copies of a single pass iterator often exposes undefined or invalid behavior if either one is incremented: the following example exposes Undefined behavior:

```
auto other = some_input_iterator;
std::cout << *(++other) << *some_input_iterator << '\n';
```

It would, therefore, make sense that classes satisfying the `InputIterator` concept shall only be required to be movable.

Alas, Single-pass iterators and many classes satisfying its requirements predate C++11, they do therefore have move only semantic with copy syntax. In that regard, they are similar to `auto_ptr`.

§ 3. Terminology

This paper redefines the requirements of some concepts as specified in the Working Draft In the rest of this paper

- `InputIterator` designs the `InputIterator` concept as proposed by this paper
- `Cpp20InputIterator` designs the `InputIterator` concept as specified in the Working Draft.
- `OutputIterator` designs the `OutputIterator` concept as proposed by this paper
- `OutputIterator` designs the `OutputIterator` concept as proposed in the Working Draft

§ 4. Scope

This paper proposes changes targeting C++20. Because the modifications proposed here changes some requirements and concepts introduced by Ranges, the authors strongly suggest they are considered for the inclusion in the same version of the standard. Indeed, Concepts introduced by ranges gives us a unique opportunity to make the modifications proposed, as they might, in some cases, break code, if introduced after the publication of C++20.

§ 4.1. Non-Goal

As a large amount of code depends on the Input/Output iterators requirements as specified by C++17, this paper does not propose any modifications to the `LegacyInputIterator` or any class that depends on it. Specifically, we do not propose to change the requirements or wording of `istream_iterator`, `ostream_iterator`, `istreambuf_iterator` or `ostreambuf_iterator`. Furthermore, we do not propose modifications to algorithms in the namespace `std`. The new iterators we propose here are in fact mostly incompatible with existing algorithms. They are meant to be used in the `ranges` namespace and as basic building blocks of range-based views.

While the ability to use move-only iterators with the algorithms defined in the `std` namespace would certainly be welcomed, doing so would weaken the `Cpp20InputIterator` concept and leads to other issues (namely, `std` based algorithms require iterators to be `EqualityComparable`, which the `Cpp20InputIterator` does not require).

In practice, that means that types satisfying the `LegacyInputIterator` requirements continue to work unaffected with algorithms defined in the `std` namespace. They may not be compatible with algorithms defined in the ranges namespace, or with new code using non-movable types satisfying the `InputIterator` concept as proposed here.

Inversely, types satisfying the `InputIterator` concepts may not be compatible with algorithms in `std` as they may not be able to satisfy the `LegacyInputIterator` requirements **if** they are not copyable.

Because it hardly makes sense to copy an Input Iterator (more on that later), it would be possible to add support for move-only iterators to the `std` namespace without much change to the standard. However, because implementers may copy iterators within the implementation of the standard library, along with existing third-party libraries, a lot of code would need to be adapted. And there is little pressure to do so as existing iterators types cannot be changed.

Furthermore, while we propose to add support for movable non-forward iterators, the proposed design does not preclude, in any way, the existence of copyable non-forward iterators.

§ 5. Motivation

§ 5.1. Move-only state

It may be desirable for an iterator to hold a move-only object, becoming itself move-only, which is not possible with iterators modeling `LegacyIterator`. A real-world example of such iterator is described in [P0902R0]. While syntactically copyable in the current design, a `coroutine_handle` such as used by a `generator` input iterator ought to be move-only.

§ 5.2. Implicitly destructive operations

Reading from an input sequence is a destructive operation. But that destruction is reflected nowhere in the API. Less experienced developers may not be aware of the destructive / single-pass nature of non-forward Iterators. By making `InputIterator` move only, developers will have to explicitly move them, which both signals the invalidation of the move-from object, but, more importantly, that the underlying data will be destroyed.

§ 6. What is a move-only iterator?

Unlike [P0902R0], we do not propose to introduce a new iterator category.

A move-only Iterator is a non-forward iterator (either input or output depending on whether it is writable). This means that a move-only iterator has *almost* the same semantic requirements as an `InputIterator`, and offers the same operations. In

other words, everything that can be expressed and done with a `Cpp20InputIterator` can be equally expressed and done with a move-only/non-copyable `InputIterator`.

Therefore, this paper does not propose to introduce a new iterator category, new named-requirement, concept name or iterator tag.

Furthermore, there is no `ForwardIterator` that is only movable, as a `ForwardIterator` is by definition an iterator that can be copied. We will expand on this later.

§ 7. A Holistic Approach to Iterators

While the first part of this paper focuses on making move-only iterators possible, as a means to get some code to compile, it is important to take a step back and to think about what movability means for Iterators, from first principles.

An iterator denotes a position into a sequence of elements (whether that sequence maps to memory or not is, for our purpose, irrelevant).

A most basic iterator can be incremented, which means it can move to the next position in the sequence. An iterator does not own the sequence iterated over (there are exceptions, ie: generators), which means the salient property of an iterator is its position in that sequence.

Iterators categories then represent the way an iterator can move along that sequence.

- `Input` and `ForwardIterator`: sequentially, one direction
- `BidirectionalIterator`: sequentially, both directions
- `RandomAccess`: both directions in $O(1)$

`ContiguousIterator` is an optimization of `RandomAccessIterator` specific to the C++ memory model that further, constrain the underlying sequence to be laid out contiguously in memory.

Stepanov theorized an additional category, "Index iterator", which has $O(1)$ access but in a single direction.

Further work was made on iterator categories, notably the `Boost.Iterator` library focused on separating traversal (how the iterator moves along the sequence) from access (whether dereferencing an iterator allows the pointed element to be read, written or both). While a very interesting concept, it falls outside the scope of this paper. Just keep in mind that everything that applies to non-forward `InputIterator` usually applies to `OutputIterator` - which are always non-Forward, the standard lacking that symmetry between read access and write access.

However, focusing on traversal, the set of iterators categories is actually rather closed, there are only so many ways a sequence can be traversed. An important point of Stepanov design is that each category is a refinement of the preceding one. `RandomAccessIterator` is a `BidirectionalIterator` which in turn is a `ForwardIterator`. Every algorithm applicable to a `ForwardIterator` can be equally applied to a `BidirectionalIterator`, etc.

So, what separates `InputIterator` from `ForwardIterator` if they are both "forward" in that they can both traverse a sequence in one direction?

`ForwardIterator` is defined as being "multi-pass". Meaning it can traverse a sequence multiple times. That, in turn, implies `ForwardIterator` is copyable, because if a sequence can be traversed multiple times, it can also be traversed multiple times at the same time and therefore there can be multiple `ForwardIterator` pointing at different elements in the sequence. `ForwardIterator` is also always `EqualityComparable`. Two `ForwardIterator` compare equal if they point to the same elements in the sequence (remember, that in the general case, the position of an iterator in a sequence is its sole salient property). And so `ForwardIterator`, being both `EqualityComparable` and `Copyable` is `Regular`.

The standard defines the "multi pass" guarantee by stating:

$a == b$ implies $++a == ++b$

Given X is a pointer type or the expression $(void)++X(a)$, a is equivalent to the expression a .

In other words: Two identical objects to which is applied the same transformation are identical. Copying a `ForwardIterator` copies the salient properties of that value and incrementing it does not modify the underlying sequence. So `ForwardIterator` is required to be a regular type behaving like a regular type.

Which bring us to `InputIterator`. `InputIterator` is a "single pass" iterator. The underlying sequence can only be traversed once. The existence of an Iterator at the n th position in the sequence implies there can be no valid iterator at the position $n-1$ in that same sequence.

```
//Given an InputIterator a
auto b = a;
a++;
b; // is invalid.
```

However, remember that the sole salient property of an iterator is its distance to the start of the sequence. Incrementing an iterator only mutates that property (again, conceptually, independently of implementation). And the only operation that mutates that property is the increment operation (which Stepanov calls `successor`).

This implies that as a non-forward iterator moves from one element of the sequence to the next, that element is destroyed.

All of this is well known and is basically rephrasing "Input iterators are single pass".

An important point to make is that how an iterator can traverse a sequence is derived from the nature of the sequence rather than from the iterator itself. The point could be made that there is no such thing as an "Input iterator" Or a "Forward Iterator" because what we really mean is "Iterator over an Input Sequence" or "Iterator over a Forward Sequence".

This is saying that, to be able to reason properly about iterators and traversal, we must assume that the iterator type associated with a sequence is the most specialized possible for that sequence.

The problem is, of course, that we do not have, in the general case, a more meaningful way to express the traversability of a sequence than by defining what type of iterator is used to iterate over it.

It is then the responsibility of the developer providing the sequence to define the most appropriate -- the most specialized -- iterator category for that sequence.

In practice, because `InputIterator` and `ForwardIterator` are syntactically identical and because of the single-pass / multi-passes guarantees are poorly taught, it is common for iterators to be mis-categorized. Other iterator categories do not have these problems as each subsequent refining category adds syntax requirements: `BidirectionalIterator` require decrement operators, `RandomAccessIterator` has further requirements. And while `ContiguousIterator` is currently not syntactically differentiated from `RandomAccessIterator`, it is possible to require that `ContiguousIterator` be convertible to a pointer of the type of the underlying sequence's elements.

But then, is there a set of operations and semantic requirements, translating to actual C++ syntax, that could allow for `InputIterator` to be easily distinguished from each other? Can we avoid requiring a tag system? Is there a defining operation that distinguishes `InputIterator` from `ForwardIterator` in such a way that it would both not require an explicit category tagging while at the same time offering a better understanding of iterator categories as well as a less surprising and safer API for non-forward iterators?

In fact, there is. We established that `ForwardIterator`s are semantically copyable, while `InputIterator`s are not. So the requirement that promotes an `InputIterator` into a `ForwardIterator` is indeed copyability - which translate in C++ to a copy constructor. We can, therefore, consider that, in the absence of a tag, all non-copyable iterators are `InputIterator`, while all copyable iterators are `ForwardIterator`.

This model, however, deviates slightly from Stepanov's work and `LegacyInputIterator`: Copying a `LegacyInputIterator` does not invalidate either copy. In fact, it is quite valid to deference multiple copies of a `LegacyInputIterator`.

Elements Of Programming has the notion of `Regular` types (and in Stepanov's work all Iterators are regular), but also the notion of regular transformations (aka pure functions) - which, given the same input, always give the same output. Given a `ForwardIterator fi`, there is a `successor` function returning an incremented copy of `fi` such as `successor(fi) == successor(fi)`. In C++, that regular `successor` function is `ForwardIterator::operator++(int)`; , in that `(it++) == (it++)` for any given `ForwardIterator`.

For `InputIterator`, Stepanov specifies that the `successor` is a pseudo transformation or a non-regular transformation that look like a regular one. And therein lies the rub.

Like a pointer, `InputIterator` is Regular, up until the point a transformation of an instance affects all copies.

```

InputIterator i = /*...*/
*i           //ok
auto a = i   //ok
*i           //ok
i++;        // a now invalid

```

This design accurately models the nature of iterators. Because an iterator represents a position in a sequence, it is natural that multiple iterators could point to the same position. After one copy is incremented, in Stepanov's model, other copies are in a partially formed state and cannot be used (but they can be assigned to, or destroyed).

Let's consider the case where we move from an iterator instead of copying it

```

InputIterator i = /*...*/
*i           //ok
auto a = move(i); //ok
*i;          //invalid
a++;         //ok
i++;         //invalid

```

Moving from an iterator invalidates it early, albeit artificially. As per standard, the moved-from iterator is in a valid, but unspecified state, and cannot be used (but can be assigned to, or destroyed). Notice the similarity between "a valid, but unspecified state" and "a partially formed state".

The difference is slim. Notably, both models are equally expressive. References can be used, should multiple names be necessary. In Stepanov's model iterators are made invalid by the natural mutation of the sequence upon increment rather than by artificially preventing multiple copies.

The second model in which the iterator is moved from, the one we think should be the default way to handle non-forward iterators, is however a much better fit for the C++ model, and offers much stronger guarantees to both the human developer as well as static analysis tools.

In the "increment invalidates" model, objects are spiritually moved-from at a distance, which neither the theory of special relativity nor the C++ memory model are equipped to handle. This makes it hard for tools to detect invalid uses - although it might become possible with better tools (See Herb Sutter's CppCon2018 talk). But most concerning, there is no way for a developer to know that the iterators are entangled.

```

auto i = troubles.begin();
auto schrodingers_iterator = i;
i++;
auto nasal_demon = *schrodingers_iterator;

```

The code above might be perfectly fine. Indeed whether it is well defined or not depends on whether the iterator return by `troubles.begin()` is forward or not. It is undecidable in these 4 lines of slide-code. It is not much more obvious in a complex program that may pass iterators to other functions or store them in containers, etc. There are, after all, no theoretical limits to the distance in time and space over which entanglement perdures.

Even worse, should the type of `troubles.begin()` be changed from Forward to Input, the code would change from perfectly fine to UB, with no warning.

Moving non-forward iterators, therefore, better expresses intent, is safer and less surprising. Move-only non-forward Iterators also express the destructive nature of incrementation and give a better sense of the difference between `InputIterator` and `ForwardIterator`.

§ 7.1. An Holistic Approach to Iterator Tags and Iterator Concepts

Missing the notion of movability pre-c++11 and lacking concepts, `LegacyIterator`s are syntactically distinguished by tags. a `LegacyInputIterator` is one which has an `input_iterator_tag` tag, while a `LegacyForwardIterator` is one which has a `forward_iterator_tag` tag. This creates a sort of circular, self-referential definition. This has carried over to the `Iterator` concepts definitions. Iterators concepts then

- Have semantic requirements not expressed through syntax and therefore not enforceable at compile time
- Need syntax to artificially subscribe to the correct, most refined concept

Of course, it is not always possible to express all of a type's semantic requirements through syntax, and in some cases, tags are an unfortunate necessity. However, they should be the mechanism of last recourse, and whenever possible, the semantic requirements should be reflected in the syntax. The idea is that hidden requirements not expressed as code lead to easier-to-misuse types, which inevitably translates to runtime bugs. **Ultimately, requirements that can neither be checked at compile time (concepts) or runtime (contracts) are bound to be ignored.** Rooted in the belief that not all birds quack like a duck, this proposal leverages meaningful syntactic requirements to increase the type safety of the iterator taxonomy.

In the case of iterators, all requirements of all iterators categories can be expressed syntactically:

```
template <class I> concept bool InputIterator =
    Readable<I> &&
    Iterator<I> ;

template <class I> concept bool ForwardIterator =
    InputIterator<I> &&
    Copyable<I> &&
    EqualityComparable<I>;

template <class I> concept bool BidirectionalIterator =
    ForwardIterator<I> &&
    Decrementable<I>;

template <class I> concept bool RandomAccessIterator =
    BidirectionalIterator<I> &&
    RandomAccessIncrementable<I>;

template <class I> concept bool ContiguousIterator =
    RandomAccessIterator<I> &&
    ConvertibleTo<I, std::add_const_t<iter_value_t<I>>*>
```

This is of course simplified but shows that each iterator category subsumes the last and adds a single, cohesive set of requirement enforceable at compile-time. In this design, there is no risk of a type satisfying the wrong concept because of a poorly chosen tag.

§ 7.1.1. Tags as an opt-in opt-out mechanism

Iterators concepts already support semantic-only checking of iterator requirements for types that do not define either `iterator_category` or iterator concept. Currently, this machinery will identify categories from `ForwardIterator` to `RandomAccessIterator`. With this proposal, non-copyable tagless types that otherwise meet the requirements of `InputIterator` be correctly identified as non-forward `InputIterator`, which is always the correct assumption. Copyable tagless iterators will remain categorized as `ForwardIterator` by that machinery.

§ 8. Q/A

§ 8.1. Non-regular iterators, really?

This proposal advocates for Non-Regular Iterators, and weakens `WeaklyIncrementable` requirements to that effect. Non-Regularity is best avoided, so this might feel like going backward.

However, **non-regular types are easier to reason about than types that just pretend to be regular.** Because `InputIterator` is meant to iterate over a non-regular sequence, it is not regular (whether we like it or not), and the best we can do is make sure the syntax matches the semantic. It would be accurate to say that `InputIterator` is locally regular, but this doesn't help much in the context of the c++ memory model. This paper is in part motivated by the conviction that exposing **a false sense of (Semi-)regularity is much more detrimental to code robustness than non-regularity.**

§ 8.2. What about Equality of Input Iterators?

A first, misguided, version of this paper attempted to prevent comparability of types meeting the `InputIterator` requirements. `InputIterator` should, in general, not be `EqualityComparable`, since they cannot be copied and a fundamental idea in Stepanov's teachings is that copy and equality are two sides of the same coin.

However, preventing `Equality` requires dramatic changes to the design and the author was reminded that negative-requirements are in general a terrible idea.

Early feedback suggested a desire to be able to compare non-forward iterators. Consider the following:

```
auto a = stream.begin();
auto b = stream.begin();
...
if(a == b) {
}
}
```

This code will inevitably lead to suffering at some point. However, we cannot prevent people from constructing multiple non-forward iterators, and these iterators will compare equal until one of them invalidate the other.

Two non-forward iterators compare equal if-and-only-if they point to the same position of the same sequence (and only one such position can be referred to at any given time).

Allowing `EqualityComparable` on non-forward iterators also simplify the interoperability of `std::` and `ranges::` iterators. However, the author would like to recommend that all future non-forward iterators introduced in the standard be *not* `EqualityComparable`. Instead, non-forward iterator should compare to a Sentinel, which is a much better model. `common_iterator` can be used to ease migration and interoperability.

§ 8.3. But... Moved-from objects are still objects!

Sure, moving-from leaves a trail of objects in an unspecified state. However, it is much more easy for tools and humans alike to understand that moved-from objects should not be used, and in fact, all majors compilers can warn about these patterns. We think that for the case at hand, focusing on the proper handling of values -- as opposed to objects -- is a sufficient approximation to reduce the potential for iterators misuse while not weakening the stronger mathematical underpinning of the STL.

§ 8.4. Does iterators default-constructability needs revisiting?

Default-constructability of iterator seems to have been added, removed and added back to the Ranges TS and the One Ranges Proposal several times. To the best of my knowledge, this was done for the sake of Semiregularity. Given that this proposal strikes semi-regularity, should this question be revisited?

The authors want to point out that default-constructed iterators are almost never in a specified state and are almost always unsafe to use. Moreover, `DefaultConstructible` is not a requirement of any algorithm using ranges and ultimately, we think enforcing `DefaultConstructibility` weakens the better `Sentinel` model introduced by ranges.

§ 8.5. What about [P0902R0]?

Andrew Hunter's "Move-only iterators" paper proposes a design to introduce Move-Only iterators in the taxonomy of `LegacyIterator`. However, this design does not offer a solution to use these move-only iterators with existing algorithms, limiting their usefulness. The iterators proposed by P0902 are additionally `EqualityComparable`. The advantage of that is that they are compatible with algorithms designed with C++17 downward. That's, however, a potential source of bugs and confusion.

However, if LEWG feels strongly about a solution compatible with existing algorithms it would be possible to relax the requirements of concerned algorithms to accept move-only iterators. along with the introduction of a new `move_iterator_tag` trait.

Such algorithms would then be compatible with types satisfying `InputIterator` (as proposed by this paper) through a `common_iterator` adaptor.

If proven with enough confidence that requirements of existing algorithms in the `std` namespace can be relaxed to handle move-only iterator, the necessary modifications can be applied in a subsequent standard version.

So while there would definitively be value in supporting move-only iterators everywhere it makes sense, and the potential for breakage is relatively low, we do not propose it for lack of visibility on the consequences of such changes.

§ 8.6. Why do you want to take my Copyable InputIterators away from me, I like them?!

We do not propose anything of the sort. But, we propose that

- Any `InputIterator` that happens to be `Copyable` is also a `ForwardIterator`.
- It remains possible to opt-out of that behavior by defining `iterator_concept` to be `input_iterator_tag`. Which has to be done in the current standard.

Non-copyable Iterator	Copyable Iterator	
<pre>struct It { It(It&&) = default; It(const It&) = delete; //... };</pre>	<pre>struct It { It(const It&) = default; //... };</pre>	<pre>struct It It(const It&) = default; using //...</pre>
<pre>static_assert<InputIterator<It>> == true; static_assert<ForwardIterator<It>> == false;</pre>	<pre>static_assert<InputIterator<It>> == true; static_assert<ForwardIterator<It>> == true;</pre>	<pre>static_ass static_ass</pre>

§ 8.7. Will this break existing code ?!

We want to reiterate(!) that all the changes proposed in this paper are only applicable to concepts, types, and requirements that were added to the standard by the Ranges proposal. They do not, in any way, impact code depending on types, requirements or algorithms as defined by the C++17 standard

§ 8.8. Won't that implicit categorization lead to miss-categorization?

The only valid use cases for `InputIterator` are streams or other input devices, and iterators that own a non-copyable generator. Most views and iterators are Forward. It turns out that C++ types are `Copyable` by default, therefore, Iterators will be categorized as `ForwardIterator` by default, which is correct in most cases.

This proposal is also a teaching opportunity because the nature of `InputIterator` is often poorly understood and misconstrued. We suspect that these tweaks to the taxonomy of Iterator will make them easier to teach.

§ 8.9. Post Increment on non-copyable iterators

Post-incrementing move-only iterators would obviously be incorrect. However, a satisfying solution was offered by [\[P0541R1\]](#).

§ 9. Questions for LEWG

- Does LEWG want to support non-copyable iterators in the `ranges` namespace?
- Does LEWG agree that **non-copyable** iterators are always non-forward iterators and do not constitute a new category (ie: no new tag should be introduced)?

- Does LEWG agree that, in the absence of an explicit tag, a **non-copyable** iterator that otherwise meets the requirement of `InputIterator`/`OutputIterator` should be recognized as such?
- Does LEWG agree that, in the absence of an explicit tag, a **copyable** iterator that otherwise meets the requirement of `ForwardIterator` should be recognized as such?
- Does LEWG want to recommend that future non-forward iterators considered for inclusion in the standard should not be copyable?
- Does LEWG want to revisit the default constructability of iterators given `Regular` is no longer a requirement of `Iterator`?
- Does LEWG want to recognize `RandomAccessIterator` providing a conversion operator to the `pointer` type of the underlying sequence as `ContiguousIterator` without the need for an explicit tag?

§ 10. List of proposed changes

Because the ranges-related proposals are still in flux and will require merging multiple documents, we do not provide wording at this time. However, a number of concepts need to be modified in order to allow for iterators that are only movable. This is a departure from the current Standard Concepts Requirements - which itself is grounded in Stepanov work - in which all iterator categories are Regular - or Semi-Regular, which implies copyability.

Note that "ForwardIterator" is defined in terms of its copyability, and so it shall remain regular. The Copyability, and therefore Regularity of Iterator is therefore moved a few levels down from `Iterator` to `ForwardIterator`

§ 10.1. Changes to `<iterator>`

§ 10.1.1. `WeaklyIncrementable`

`WeaklyIncrementable` is a requirement of all `Iterator`, including `Cpp20InputIterator`. `WeaklyIncrementable` is defined to be semi-regular. Because `WeaklyIncrementable`, as it is described in [P0896R2], accommodates for `Cpp20InputIterator` and `LegacyInputIterator`, it suffers from the same issue (being copyable with move semantic). We propose to strike the `Semiregular` requirement as follow

```
template <class I>
concept WeaklyIncrementable =
    Movable<I> &&
    requires(I i) {
        typename iter_difference_t<I>;
        requires SignedIntegral<iter_difference_t<I>>;
        { ++i } -> Same<I>&; // not required to be equality-preserving
        i++; // not required to be equality-preserving
    };
```

§ 10.1.2. `Iterator`

`Iterator` is left unmodified as merely changing `WeaklyIncrementable` is enough to not requiring it to be regular.

§ 10.1.3. `InputIterator`

The `InputIterator` iterator concept is left unchanged. It implicitly doesn't need to meet the requirement of `Semiregular` any longer through `WeaklyIncrementable`.

Tagless move-only input iterators will also be correctly identified by the `ITER_CONCEPT` machinery without further changes.

§ 10.1.4. OutputIterator

The `OutputIterator` iterator concept is left unchanged. It implicitly doesn't need to meet the requirement of `Semiregular` any longer through `WeaklyIncrementable`.

§ 10.1.5. ContiguousIterator

```
template <class I, class T>
concept ContiguousIterator =
    ContiguousIterator =
        RandomAccessIterator<I> &&
        std::is_lvalue_reference<iter_reference_t<I>>::value &&
        Same<iter_value_t<I>, __uncvref<iter_reference_t<I>>> &&
        (
            DerivedFrom<ITER_CONCEPT(I), contiguous_iterator_tag> ||
            ConvertibleTo<I, std::add_const_t<iter_value_t<I>>*>
        );
```

Requiring a conversion operator to `iter_value_t<I>*` (instead of a tag - It could also require a tag **or** a conversion operator) allows the whole iterator taxonomy to be semantically and accurately constrainable without the need for tags.

§ 10.2. Changes to <ranges>

§ 10.2.1. Views

Unlike R0, this paper does not propose to remove the `SemiRegular` requirement from view. View over non-forward ranges should not hold or cache the result of calling `begin()`

§ 10.2.2. Inserters

Because the `OutputIterator` concept as proposed here is not compatible with the `LegacyOutputIterator` requirements, it would not be possible to use `std::` inserters with the `ranges::` algorithms.

It is, therefore, necessary to provide suitable inserters modeling `OutputIterator`

§ 10.2.2.1. back_insert_iterator

```
namespace std::ranges {
    template <class Container>
    class back_insert_iterator : public std::back_insert_iterator<Container> {
    public:
        using std::back_insert_iterator<Container>::back_insert_iterator;
        back_insert_iterator(const back_insert_iterator & other) = delete;
        back_insert_iterator(back_insert_iterator && other)

    };
    template <class Container>
    back_insert_iterator<Container> back_inserter(Container& x);
}
```

§ 10.2.2.2. *front_insert_iterator*

```
namespace std::ranges {  
    template <class Container>  
    class front_insert_iterator : public std::front_insert_iterator<Container> {  
    public:  
        using std::front_insert_iterator<Container>::front_insert_iterator;  
        front_insert_iterator(const front_insert_iterator & other) = delete;  
        front_insert_iterator(front_insert_iterator && other);  
    };  
    template <class Container>  
    front_insert_iterator<Container> front_inserter(Container& x);  
}
```

§ 10.2.2.3. *insert_iterator*

```
namespace std::ranges {  
    template <class Container>  
    class insert_iterator : public std::insert_iterator<Container> {  
    public:  
        using std::insert_iterator<Container>::insert_iterator;  
        insert_iterator(const insert_iterator & other) = delete;  
        insert_iterator(insert_iterator && other);  
    };  
    template <class Container>  
    insert_iterator<Container> inserter(Container& x, typename Container::iterator i);  
}
```

§ 10.2.3. **counted_iterator**

- `counted_iterator` copy constructor needs to be disabled for non-copyable iterators
- `counted_iterator::base()` needs to be disabled for non-copyable iterators

§ 10.2.4. **move_iterator**

- `move_iterator` copy constructor needs to be disabled for non-copyable iterators
- `move_iterator::base()` needs to be disabled for non-copyable iterators

§ 10.3. Changes to `<algorithm>`

There should be no wording change to algorithms. However, implementers would have to never copy non-forward iterators within the implementation of C++20 ranges based and ranges:: algorithms.

§ 11. Impact on other proposals

§ 11.1. View proposals

We suggest that the various proposals introducing new non-forward views should have iterators that are neither copyable nor equally comparable. A candidate for such change would be [P1035R1](#)'s `istream_view`.

§ 11.2. Iterator facade

[P0186R0] describes a system for an iterator facade. The changes proposed make defining iterators easier but we think there is still value in an iterator facade. To accommodate and honor the changes proposed here, we suggest that:

- `Cursor` should not be required to be copyable, nor should be `mixin_t`
- An iterator constructed from a move-only `Cursor` models an `InputIterator` (or `OutputIterator` depending on whether a `write` member function is defined)
- An iterator facade constructed from a Copyable cursor with an `equal(const cursor &)` member function models an `ForwardIterator`, unless it defines a `single_pass = true` trait.
- `cursor::Forward` should require `Semi-regularC> && !C::single_pass`
- `cursor::Contiguous` should require something along the line of `ConvertibleTo<value_type_t<C>*, C> || C::contiguous;`

§ 12. Implementation experience

We validated the design in `cmstl2`. However, `cmstl2` deviates from the Working Draft as it doesn't have the same Deep Integration system and therefore lacks the `ITER_CONCEPT` machinery. Furthermore, we have not yet completed this work. Some algorithms, like `count`, proved to require specialization for `InputIterator` because of implementation specific details.

§ 12.1. Acknowledgments

The authors like to thank Connor Waters, Tony Van Eerd, Eric Niebler, Casey Carter, Christopher Di Bella, Sean Parent and Arthur O'Dwyer who gave tremendously helpful feedbacks during the writing of this paper.

§ References

§ Informative References

[P0186R0]

Beman Dawes, Eric Niebler, Casey Carter. [Iterator Facade Library Proposal for Ranges](https://wg21.link/p0186r0), 11 February 2016. URL: <https://wg21.link/p0186r0>

[P0541R1]

Eric Niebler. [Ranges TS: Post-Increment on Input and Output Iterators](https://wg21.link/p0541r1), 10 July 2017. URL: <https://wg21.link/p0541r1>

[P0896R2]

Eric Niebler, Casey Carter, Christopher Di Bella. [The One Ranges Proposal](https://wg21.link/p0896r2), 25 June 2018. URL: <https://wg21.link/p0896r2>

[P0902R0]

Andrew Hunter. [Move-only iterators](https://wg21.link/p0902r0), 5 February 2018. URL: <https://wg21.link/p0902r0>

[P1035R1]

Christopher Di Bella. [Input range adaptors](https://wg21.link/p1035). URL: <https://wg21.link/p1035>